



JAVA PROGRAMMING • STUDY GUIDE

LOOPS AND PATTERNS

Complete Study Guide with Examples, Dry Runs & Flowcharts

For Loop

While Loop

Nested Loops

Pattern Programs

```
for(int i = 1; i <= n; i++) {  
    for(int j = 1; j <= i; j++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
}
```

B.Tech – Artificial Intelligence & Machine Learning

Karunya Institute of Technology and Sciences

Batch 2023 – 2027

Table of Contents

1 Introduction to Loops and Patterns

2 The For Loop

3 The While Loop

4 For Loop vs While Loop

5 Nested Loops

6 Pattern Programming

7 Quick Revision Sheet

How to use this guide

Each topic follows the same structure: concept → syntax → flowchart → worked examples with dry runs → advantages/applications → key points → summary. Work through the dry-run tables by hand before checking the output — that's what builds real intuition for loops.

In programming, many situations require a set of statements to be executed repeatedly. Writing the same code multiple times makes programs lengthy and difficult to maintain. To solve this problem, Java provides **loops**, which allow a block of code to run multiple times based on a condition.

The most commonly used loops in Java are the **for** loop and the **while** loop. They help automate repetitive tasks such as processing data, performing calculations, and traversing collections of elements.

Sometimes one loop needs to run inside another loop to handle more complex operations. Such loops are called **nested loops**, and they are especially useful for working with tables, matrices, and generating output patterns.

Using loops, programmers can also create **patterns** — arrangements of numbers, characters, or symbols displayed in a specific format. Pattern programs improve logical thinking and help build a solid intuition for how loops work together.

Real-life analogy

Think of a loop as a washing machine cycle: you set a condition ("spin until the timer hits zero"), and the machine keeps repeating the wash-rinse-spin steps automatically, instead of you doing each step by hand every single time.

Topics Covered

- **For Loop** — used when the number of iterations is known in advance.
- **While Loop** — used when the number of iterations is not known in advance.
- **Nested Loops** — a loop placed inside another loop.
- **Patterns** — programs that use loops to print shapes, numbers, or character arrangements.

A First Look: Pattern from Nested Loops

OUTPUT

```
*****  
*****  
*****
```

This pattern is generated using nested loops, where the **outer loop** controls the number of rows and the **inner loop** controls the number of columns. Understanding this row-column relationship is the key to solving almost every pattern problem you will encounter.

The **for** loop is one of the most commonly used looping statements in Java. It executes a block of code repeatedly for a specified number of times, and is preferred whenever the number of iterations is known in advance. Instead of writing the same statements multiple times, a for loop lets you write concise, efficient code.

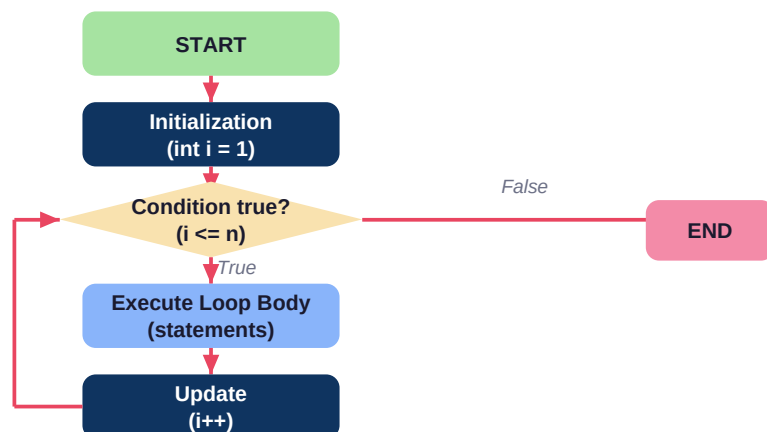
Syntax

```
1 for(initialization; condition; update)
2 {
3     // statements
4 }
```

Components of a For Loop

- **Initialization** — executed only once, at the very beginning; declares/initializes the loop control variable.
- **Condition** — evaluated before every iteration; loop body runs while it is true, and the loop ends the moment it is false.
- **Update** — executed after each iteration; usually increments or decrements the loop variable.

Working of the For Loop



- Initialization is executed (once).
- Condition is checked.
- If true → the loop body executes.
- Update statement executes.
- Control returns to the condition, and steps 2–4 repeat until the condition becomes false.

Example 1: Print Numbers from 1 to 5

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 5; i++) {
4             System.out.println(i);
5         }
6     }
7 }
```

OUTPUT

```
1
2
3
4
5
```

Dry Run

Iteration	i (value)	Condition (i <= 5)	Output
1	1	True	1
2	2	True	2
3	3	True	3
4	4	True	4
5	5	True	5
6	6	False	Loop Ends

Example 2: Print Even Numbers from 2 to 10

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 2; i <= 10; i += 2) {
4             System.out.println(i);
5         }
6     }
7 }
```

OUTPUT

```
2
4
6
8
10
```

Example 3: Print Numbers in Reverse Order

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 5; i >= 1; i--) {
4             System.out.println(i);
5         }
6     }
7 }
```

OUTPUT

```
5
4
3
2
1
```

Example 4: Sum of First 5 Natural Numbers

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         int sum = 0;
4         for(int i = 1; i <= 5; i++) {
5             sum = sum + i;
6         }
7         System.out.println("Sum = " + sum);
8     }
9 }
```

OUTPUT

```
Sum = 15
```

Dry Run

i	sum
1	1
2	3
3	6
4	10
5	15

Example 5: Multiplication Table of 5

```
Main.java Java
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 10; i++) {
4             System.out.println("5 x " + i + " = " + (5 * i));
5         }
6     }
7 }
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Advantages of For Loop

- ✓ Simple and easy to understand.
- ✓ Suitable when the number of iterations is known.
- ✓ Reduces code repetition.
- ✓ Improves program readability.
- ✓ Makes code easier to maintain.

Applications

- Printing sequences of numbers.
- Calculating sums and products.
- Generating multiplication tables.
- Traversing arrays and collections.
- Creating patterns.

The **for** loop is a repetition control structure that executes a block of code multiple times, combining initialization, condition checking, and update in a single line. It is most suitable when the number of iterations is known beforehand and is widely used in Java for solving repetitive tasks efficiently.

3

The While Loop

Best suited when the number of iterations is unknown

The **while** loop repeatedly executes a block of code as long as a specified condition remains true. It is commonly used when the number of iterations is not known beforehand. Before each iteration the condition is checked — if true the loop body executes, if false the loop terminates. Because the condition is tested *before* entering the loop body, the while loop is called an **entry-controlled loop**.

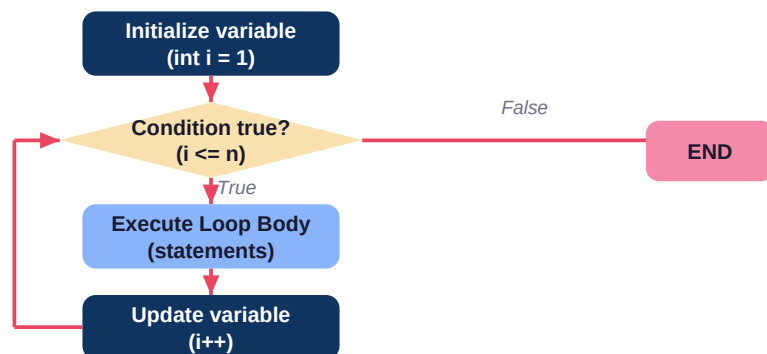
Syntax

```
1 while(condition)
2 {
3     // statements
4 }
```

Components of While Loop

- **Condition** — evaluated before each iteration; must return true or false.
- **Loop Body** — statements executed repeatedly while the condition remains true.
- **Update Statement** — usually placed inside the body; updates the loop control variable to avoid infinite execution.

Working of the While Loop



Example 1: Print Numbers from 1 to 5

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         int i = 1;
4         while(i <= 5) {
5             System.out.println(i);
6             i++;
7         }
8     }
9 }
```

OUTPUT

```
1
2
3
4
5
```

Dry Run

Iteration	i (value)	Condition (i <= 5)	Output
1	1	True	1
2	2	True	2
3	3	True	3
4	4	True	4
5	5	True	5
6	6	False	Loop Ends

Example 2: Print Even Numbers from 2 to 10

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         int i = 2;
4         while(i <= 10) {
5             System.out.println(i);
6             i += 2;
7         }
8     }
9 }
```

OUTPUT

```
2
4
6
8
10
```

Example 3: Print Numbers in Reverse Order

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         int i = 5;
4         while(i >= 1) {
5             System.out.println(i);
6             i--;
7         }
8     }
9 }
```

OUTPUT

```
5
4
3
2
1
```

Example 4: Sum of First 5 Natural Numbers

Main.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         int i = 1;
4         int sum = 0;
5         while(i <= 5) {
6             sum = sum + i;
7             i++;
8         }
9         System.out.println("Sum = " + sum);
10    }
11 }
```

OUTPUT

```
Sum = 15
```

Dry Run

i	sum
1	1
2	3
3	6
4	10
5	15

Example 5: Multiplication Table of 5

```
Main.java Java
1 public class Main {
2     public static void main(String[] args) {
3         int i = 1;
4         while(i <= 10) {
5             System.out.println("5 x " + i + " = " + (5 * i));
6             i++;
7         }
8     }
9 }
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

■ The Infinite While Loop Trap

If the loop variable is not updated properly, the condition may never become false, resulting in an infinite loop that never terminates.

```
InfiniteLoop.java Java
1 int i = 1;
2 while(i <= 5) {
3     System.out.println(i);
4 } // i is never incremented!
```

OUTPUT

```
1
1
1
1
...
```

Golden rule

Every while loop needs a way out. Always double-check that the update statement inside the body can actually make the condition false — otherwise the loop runs forever.

Advantages of While Loop

- ✓ Suitable when the number of iterations is unknown.
- ✓ Simple and easy to use.
- ✓ Useful for input validation and menu-driven programs.
- ✓ Provides flexibility in controlling loop execution.

Applications

- Reading user input until a valid value is entered.
- Menu-driven applications.
- Repeating tasks based on conditions.
- Processing data until a specific condition is met.
- Game and simulation programs.

The **while** loop is an entry-controlled looping statement that executes a block of code repeatedly as long as a specified condition remains true. It is particularly useful when the number of iterations is not known in advance, and it provides flexibility for condition-based repetition in Java programs.

4

For Loop vs While Loop

Choosing the right tool for the job

For Loop	While Loop
Used when the number of iterations is known .	Used when the number of iterations is unknown .
Initialization, condition, and update are written together.	Initialization and update are written separately.
More compact syntax.	More flexible for condition-based repetition.
May execute zero or more times.	May execute zero times if the condition is initially false.
Preferred for counting / range-based tasks.	Preferred for input validation and event-driven tasks.

Quick decision rule

Ask yourself: “Do I already know exactly how many times this needs to repeat?” If yes → reach for a **for** loop. If the repetition depends on something that changes at runtime (like user input) → reach for a **while** loop.

5

Nested Loops

A loop placed inside another loop

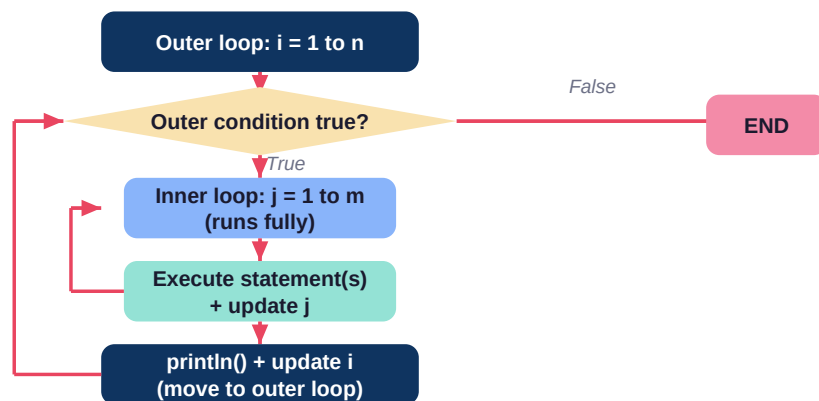
A **nested loop** is a loop placed inside another loop. The outer loop controls how many times the inner loop executes. For every single iteration of the outer loop, the inner loop completes *all* of its iterations. Nested loops are commonly used for working with tables, matrices, and pattern-printing programs. Java allows any type of loop (for, while, or do-while) to be nested inside another loop.

Syntax

```
1 for(initialization1; condition1; update1)
2 {
3     for(initialization2; condition2; update2)
4     {
5         // statements
6     }
7 }
```

Working of Nested Loops

- The outer loop starts its first iteration.
- The inner loop executes completely.
- The outer loop moves to the next iteration.
- The inner loop again executes completely.
- This continues until the outer loop's condition becomes false.



Example 1: Rectangle of Stars

```
Main.java Java
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 3; i++) {
4             for(int j = 1; j <= 5; j++) {
5                 System.out.print("* ");
6             }
7             System.out.println();
8         }
9     }
10 }
```

OUTPUT

```
* * * * *
* * * * *
* * * * *
```

Outer loop controls rows, inner loop controls columns: 3 rows and 5 columns of stars are printed.

Example 2: Multiplication Table (5×5 grid)

```
Main.java Java
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 5; i++) {
4             for(int j = 1; j <= 5; j++) {
5                 System.out.print(i * j + "\t");
6             }
7             System.out.println();
8         }
9     }
10 }
```

OUTPUT

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
```

Dry Run Example

Snippet.java

Java

```
1 for(int i = 1; i <= 2; i++) {
2     for(int j = 1; j <= 3; j++) {
3         System.out.print("* ");
4     }
5     System.out.println();
6 }
```

Outer loop (i)	Inner loop (j)	Row output
1	1, 2, 3	* * *
2	1, 2, 3	* * *

OUTPUT

```
* * *
* * *
```

Common Uses of Nested Loops

1. Pattern Printing

Pattern.java

Java

```
1 for(int i = 1; i <= 4; i++) {
2     for(int j = 1; j <= 4; j++) {
3         System.out.print("* ");
4     }
5     System.out.println();
6 }
```

2. Matrix Operations

Matrix.java

Java

```
1 for(int row = 0; row < 3; row++) {
2     for(int col = 0; col < 3; col++) {
3         System.out.print(matrix[row][col] + " ");
4     }
5     System.out.println();
6 }
```

3. Multiplication Tables (1–10)

TimesTable.java

Java

```
1 for(int i = 1; i <= 10; i++) {
2     for(int j = 1; j <= 10; j++) {
3         System.out.print(i * j + "\t");
4     }
5     System.out.println();
6 }
```


Advantages

- Useful for handling rows and columns.
- Helps in matrix and table operations.
- Essential for pattern printing.
- Simplifies complex repetitive tasks.
- Improves logical thinking and problem-solving skills.

Disadvantages

- Can increase program execution time.
- Difficult to understand if deeply nested.
- Excessive nesting may reduce code readability.

Key Points to Remember

- ✓ A loop inside another loop is called a nested loop.
- ✓ The inner loop executes completely for each iteration of the outer loop.
- ✓ Outer loop generally controls rows; inner loop generally controls columns.
- ✓ Nested loops are widely used in pattern and matrix problems.

A **nested loop** is a loop placed inside another loop. The outer loop controls the overall repetitions, while the inner loop performs repeated operations for each outer loop iteration. Nested loops are commonly used in Java for pattern printing, matrix processing, table generation, and other row-column based operations.

Pattern programming is the process of printing numbers, characters, or symbols in a specific arrangement using loops. Pattern programs are one of the best ways to understand how loops work together, especially nested loops. Most pattern programs follow the same structure: the **outer loop controls the number of rows**, and the **inner loop controls the number of columns** or elements printed in each row.

Why Learn Pattern Programming?

- Improves understanding of loops.
- Strengthens logical thinking.
- Helps in coding interviews and programming tests.
- Builds a strong foundation for solving complex problems.
- Enhances problem-solving skills.

Types of Patterns

- Star Patterns (*)
- Number Patterns
- Alphabet Patterns
- Pyramid Patterns
- Inverted Patterns

Golden template

```
for(row = 1; row <= n; row++) { for(column = 1; column <= requiredValue; column++) { /* print symbol */ }  
System.out.println(); } — Once you can name what requiredValue is for a row, you can build almost any pattern.
```

1. Square Star Pattern

OUTPUT

```
* * * *  
* * * *  
* * * *  
* * * *
```

Square.java

Java

```
1 public class Main {  
2     public static void main(String[] args) {  
3         for(int i = 1; i <= 4; i++) {  
4             for(int j = 1; j <= 4; j++) {  
5                 System.out.print("* ");  
6             }  
7             System.out.println();  
8         }  
9     }  
10 }
```

Outer loop prints 4 rows; inner loop prints 4 stars in each row.

2. Right Triangle Star Pattern

OUTPUT

```
*
* *
* * *
* * * *
```

RightTriangle.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 4; i++) {
4             for(int j = 1; j <= i; j++) {
5                 System.out.print("* ");
6             }
7             System.out.println();
8         }
9     }
10 }
```

The row number determines how many stars are printed — the count of stars increases in each row (inner loop runs from 1 to i).

3. Inverted Triangle Pattern

OUTPUT

```
* * * *
* * *
* *
*
```

InvertedTriangle.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 4; i >= 1; i--) {
4             for(int j = 1; j <= i; j++) {
5                 System.out.print("* ");
6             }
7             System.out.println();
8         }
9     }
10 }
```

The outer loop counts down, so the number of stars decreases row by row.

4. Number Triangle Pattern

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
```

NumberTriangle.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 4; i++) {
4             for(int j = 1; j <= i; j++) {
5                 System.out.print(j + " ");
6             }
7             System.out.println();
8         }
9     }
10 }
```

Each row prints numbers from 1 up to the row number — notice the inner loop prints *j*, not a fixed symbol.

5. Repeated Number Pattern

OUTPUT

```
1
2 2
3 3 3
4 4 4 4
```

RepeatedNumber.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 4; i++) {
4             for(int j = 1; j <= i; j++) {
5                 System.out.print(i + " ");
6             }
7             System.out.println();
8         }
9     }
10 }
```

Almost identical to Pattern 4, but the row number *i* is printed repeatedly instead of the counter *j*.

6. Alphabet Triangle Pattern

OUTPUT

```
A
A B
A B C
A B C D
```

AlphabetTriangle.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         for(int i = 1; i <= 4; i++) {
4             for(char ch = 'A'; ch < 'A' + i; ch++) {
5                 System.out.print(ch + " ");
6             }
7             System.out.println();
8         }
9     }
10 }
```

The inner loop treats a char like a number, walking from 'A' up to 'A'+i so each row adds one more letter of the alphabet.

7. Pyramid Star Pattern

OUTPUT

```
  *
 * * *
* * * * *
* * * * * * *
```

Pyramid.java

Java

```
1 public class Main {
2     public static void main(String[] args) {
3         int n = 4;
4         for(int i = 1; i <= n; i++) {
5             for(int j = 1; j <= n - i; j++) {
6                 System.out.print(" ");
7             }
8             for(int k = 1; k <= (2 * i - 1); k++) {
9                 System.out.print("* ");
10            }
11            System.out.println();
12        }
13    }
14 }
```

- The **first inner loop** prints leading spaces to centre the shape.
- The **second inner loop** prints $(2i-1)$ stars — the odd-number trick that keeps a pyramid symmetric.
- Together they create a centred pyramid shape.

Dry Run Example

```
Snippet.java Java
1 for(int i = 1; i <= 3; i++) {
2     for(int j = 1; j <= i; j++) {
3         System.out.print(" * ");
4     }
5     System.out.println();
6 }
```

Row (i)	Stars Printed
1	*
2	* *
3	* * *

```
OUTPUT
*
* *
* * *
```

Rule of Thumb

- ✓ Outer loop → Rows.
- ✓ Inner loop → Columns / elements.
- ✓ `println()` moves to the next line.
- ✓ `print()` keeps output on the same line.

Applications of Pattern Programming

- Learning nested loops.
- Improving logical reasoning.
- Coding interview preparation.
- Competitive programming.
- Understanding matrix-like structures.

Pattern programming is the technique of printing symbols, numbers, or characters in structured formats using loops. It helps master nested loops, develops logical thinking, and improves problem-solving skills. Most patterns are created using an outer loop for rows and an inner loop for columns, making pattern programming an essential topic in Java programming.

Core Definitions

Term	Meaning
for loop	Repeats a block a known number of times; init/condition/update together.
while loop	Repeats a block while a condition is true; entry-controlled, count often unknown.
Nested loop	A loop inside another loop; inner completes fully per outer iteration.
Pattern program	Uses nested loops to print rows/columns of symbols in a fixed shape.

Pattern-Building Cheat Sheet

Pattern	Inner loop range	What's printed
Square	$j = 1$ to n (fixed)	"* "
Right triangle	$j = 1$ to i	"* "
Inverted triangle	$j = 1$ to i (i counts down)	"* "
Number triangle	$j = 1$ to i	j
Repeated number	$j = 1$ to i	i
Alphabet triangle	$ch = 'A'$ to $'A'+i$	ch
Pyramid	spaces: $n-i$, stars: $2i-1$	" " then "*" "

Common Mistakes to Avoid

- ✓ Forgetting to update the loop variable in a while loop (infinite loop).
- ✓ Using `print()` instead of `println()` and breaking the row structure.
- ✓ Mixing up i (rows) and j (columns) inside nested loops.
- ✓ Off-by-one errors in the condition ($\leq$ vs $<$).
- ✓ Forgetting the space/tab needed to separate printed values.

Master the four building blocks in order: get comfortable with a single **for** loop, then a single **while** loop, then combine two loops into a **nested loop**, and finally use that nested structure to build **patterns**. Every pattern program is just a for-loop skeleton with a smarter inner-loop range.